

CAN Bus Intrusion Detection System

Daniel Lee

May 2026

Contents

1	Introduction	2
2	System Design	2
3	Implementation	3
4	Evaluation	4
5	Limitations	5
6	Future Work	5
7	Conclusion	5

1 Introduction

The Controller Area Network (CAN) protocol is a lightweight communication standard used in virtually all modern vehicles to connect electronic control units (ECUs). Despite its widespread adoption, CAN has no built-in authentication or encryption, making it vulnerable to attacks such as message flooding, spoofing, and data injection. Real-world exploits have demonstrated the severity of this vulnerability. In 2015, security researchers remotely compromised a Jeep Cherokee via its CAN bus, gaining control of the brakes and steering at highway speeds [1]. This project implements a rule-based Intrusion Detection System (IDS) for CAN bus networks, capable of detecting these attack types in real time, and evaluates it against the OTIDS dataset, a publicly available benchmark of real CAN attack traffic captured from a Kia Soul [2].

2 System Design

For the detection approach, the project is rule-based rather than machine-learning (ML) based. There are three reasons as to why the rule-based approach was selected for the project despite the ML approach being a more accurate option. First, the rule-based approach initially seemed to be able to cover flooding, spoofing, and payload attacks, which are the attacks I had selected to cover. Secondly, this approach allowed for deterministic behavior, allowing for less errors in implementation. Lastly, the project had a 10-week deadline, so the ML approach including research, implementation, validation, and documentation was not guaranteed viable. These results highlight that ML-based approaches would be better suited for impersonation detection in future iterations.

Three attack types were selected for detection: DoS flooding, ID spoofing, and payload anomalies. These were chosen because they are the most well-documented CAN attacks in literature, can be simulated and evaluated using publicly available datasets, and collectively represent the majority of real-world CAN attack scenarios. Timing and bus-off attacks were excluded due to implementation complexity and hardware requirements beyond the scope of this project.

At a high level overview, the system architecture is organized into three layers. The data layer consists of the Message class, which represents a single CAN message with an arbitration ID, payload, timestamp, and an optional extended flag parameter that is not utilized for the OTIDS dataset. The detection layer contains three independent rules — FloodingRule, SpoofingRule, and PayloadRule — each implementing a check() method that returns an alert string or None. The coordinator (Detector) holds a list of all rules and runs each incoming message through them, collecting any alerts fired. A TrafficGenerator was also implemented to simulate normal and attack traffic for unit testing, and a loader module parses the OTIDS CSV dataset into Message objects for real-world evaluation.

3 Implementation

The IDS implementation consists of three independent detection rules, each targeting a specific attack pattern. Rules are designed to be stateless where possible, with a common `check(message)` interface that returns an alert string when an anomaly is detected or `None` otherwise.

`FloodingRule` detects DoS flooding attacks by monitoring the message frequency of each CAN arbitration ID using a sliding window approach. For each incoming message, the rule maintains a list of recent timestamps per ID and discards any timestamps older than one second. If the count of remaining timestamps exceeds a configurable threshold, an alert is raised. The threshold of 125 messages/second was derived by computing the maximum observed frequency of any single ID in the OTIDS attack-free dataset (124 msgs/sec), ensuring normal traffic never triggers the rule while flood traffic, which reaches 335,000 msgs/sec for ID 0x000, is reliably detected. Note that ID 0x000 is the attack ID for DoS attacks in the OTIDS dataset [2].

`SpoofingRule` detects two forms of spoofing. First, whitelist validation flags any message whose arbitration ID is not in the set of known legitimate IDs, derived from the attack-free dataset. Second, duplicate sender detection flags consecutive messages sharing the same ID that arrive within 1ms of each other — a gap too small for a single ECU to produce, indicating a second device impersonating the same node. The 1ms threshold was chosen based on the minimum observed inter-message gap in normal traffic for monitored IDs. This implementation came with limitations when using the OTIDS dataset.

`PayloadRule` validates message payloads against a configurable spec that maps each CAN ID to an expected byte range and valid value bounds. For each incoming message, the rule checks that the payload is long enough to contain the specified bytes, then extracts those bytes as an integer and verifies the value falls within the defined minimum and maximum. For example, a spec entry for a speed signal might define bytes 0-1 as a value between 0 and 200 km/h; any message exceeding that range is flagged. A default spec with placeholder values was used for unit testing; real-world deployment would require a vehicle-specific Database CAN (DBC) file to define accurate signal ranges. This implementation also came with limitations when using the OTIDS dataset.

4 Evaluation

The system was evaluated against the OTIDS Car Hacking Dataset [2], captured from a Kia Soul and containing four labeled CSV files: attack-free traffic, DoS flooding, fuzzy attacks, and impersonation attacks. Prior to evaluation, the attack-free dataset was used to derive two parameters: the whitelist of legitimate CAN IDs (45 unique IDs) for SpoofingRule, and the maximum normal message frequency (124 msgs/sec) used to set the FloodingRule threshold at 125 msgs/sec. The impersonation dataset labels all rows as target=3 including an attack-free period in the first 250 seconds; these rows were excluded from evaluation to avoid inflating false negatives.

Attack Type	Detection Rate	Notes
DoS (Flooding)	100%	FloodingRule — ID 0x000 floods at 335,624 msgs/sec
Fuzzy (Spoofing)	~100%	SpoofingRule — random IDs caught by whitelist check
Impersonation	~0%	Attacker mimics normal rate (130 vs 124 msgs/sec)
Overall	72.27%	Across all attack datasets

Table 1: Detection results on OTIDS dataset

False positive rate of 4.70% was measured against the attack-free dataset. Throughput was measured at 35,627 messages/second, exceeding the real CAN bus rate of 2,000–10,000 msgs/second and confirming real-time detection capability.

PayloadRule could not be meaningfully evaluated on this dataset as the OTIDS dataset does not include a DBC file specifying signal definitions for each CAN ID. Without known byte positions and valid value ranges per ID, payload validation cannot be applied. The rule was verified through unit testing with a synthetic spec instead.

DoS detection achieved 100% because the flooding rate (335,624 msgs/sec) is orders of magnitude above the threshold. Fuzzy attack detection is high because random CAN IDs are almost always absent from the whitelist. Impersonation detection failed because the attacker sends on a legitimate ID (0x164) at only 130 msgs/sec — too close to the normal rate of 124 msgs/sec to distinguish with frequency-based rules. This is a fundamental limitation of rule-based detection, which will be discussed in more detail.

5 Limitations

Rule-based detection has fundamental constraints that became apparent during evaluation. First, impersonation attacks that mimic normal message rates cannot be detected with frequency-based rules — the attacker sends on ID 0x164 at 130 msgs/sec while the legitimate ECU sends at 124 msgs/sec, a difference too small to distinguish without timing fingerprinting. Second, payload validation requires a vehicle-specific DBC file to define signal ranges per CAN ID; the OTIDS dataset does not include one for the Kia Soul, so PayloadRule could not be evaluated on real attack data and was verified through unit testing only. Third, the flooding detection threshold of 125 msgs/sec was derived empirically from this dataset and may require recalibration for vehicles with different normal message rates. Finally, the impersonation dataset labels all rows — including the first 250 seconds of normal traffic — as target=3, requiring manual timestamp filtering to produce accurate detection metrics.

6 Future Work

Several improvements are planned for future iterations of this system. The most impactful would be clock-skew detection; each ECU’s crystal oscillator drifts at a unique rate, so tracking this drift over time from normal traffic would allow the system to flag messages from an impersonating device even when it sends at a normal rate. This approach would directly address the impersonation detection gap identified in evaluation. A machine learning approach, such as an LSTM trained on normal CAN traffic sequences, could also capture subtle behavioral patterns that fixed rules cannot. Sourcing or reverse-engineering a DBC file for the target vehicle would enable meaningful payload validation, completing the third detection rule. Finally, a C/C++ port of the core detection logic would allow deployment on embedded vehicle hardware with minimal memory and processing overhead, moving the system closer to a production-ready implementation.

7 Conclusion

This project implemented a rule-based CAN bus Intrusion Detection System capable of detecting DoS flooding, ID spoofing, and payload anomalies in real time. Evaluated against the OTIDS dataset captured from a Kia Soul, the system achieved 100% detection rate on DoS attacks and approximately 100% on fuzzy attacks, with an overall detection rate of 72.27% and a false positive rate of 4.70%. Throughput of 35,627 messages/second confirms real-time detection capability well above actual CAN bus rates. Impersonation detection proved fundamentally limited by the nature of rule-based approaches — the attacker mimics normal traffic rates closely enough that frequency rules cannot distinguish it without timing fingerprinting. While the 90% detection target was not met overall, the results demonstrate that rule-based detection is highly effective for high-frequency attacks and provides a solid foundation for future improvements including clock-skew analysis and machine learning integration.

References

- [1] C. Miller and C. Valasek, “Remote Exploitation of an Unaltered Passenger Vehicle,” DEF CON 23, 2015.
- [2] H. Lee, S. H. Jeong, and H. K. Kim, “OTIDS: A Novel Intrusion Detection System for In-vehicle Network by using Remote Frame,” in *Proc. PST*, 2017.